ESPRIT

Architecture web distribuée

Classe : 4DAS_I13

RAPPORT TECHNIQUE DE CONCEPTION

Conception d'une plateforme e-learning

Architecture distribuée par microservices

Validation intermédiaire

Équipe du projet

Membre	Responsabilité principale
Maram Dhambri	Gestion des cours
Badis Awedi	Gestion des quiz et examens
Aymen Ben Salem	Gestion des réclamations
Dheker Derouiche	Gestion des classes
Saif Dkhaili	Gestion des paiements
Souleimen Belhadj	Gestion du forum

Enseignant du module :
Oussema Sellami

Année universitaire :
2026–2025

Le module Utilisateurs / Authentification est pris en charge en commun par toute l'équipe.

Table des matières

Liste des figures **iv**

Liste des tableaux **v**

Introduction **1**

1 Présentation générale du projet **2**

1.1 Contexte 2

1.2 Problématique 2

1.3 Solution proposée 2

1.4 Objectifs du projet 3

1.5 Périmètre du projet 3

2 Analyse des besoins **4**

2.1 Introduction 4

2.2 Acteurs du système 4

2.3 Besoins fonctionnels 4

2.3.1 Authentification et gestion des utilisateurs 4

2.3.2 Gestion des cours 5

2.3.3 Gestion des classes 5

2.3.4 Gestion des quiz et examens 5

2.3.5 Gestion des réclamations 5

2.3.6 Gestion des paiements 5

2.3.7 Gestion du forum 5

2.4 Besoins non fonctionnels 5

2.5 Diagramme global de cas d'utilisation 6

2.6 Répartition des responsabilités 7

2.7 Conclusion 7

3 Architecture globale **8**

3.1 Introduction 8
3.2 Principes de l'architecture 8
3.3 Diagramme d'architecture globale 8
3.4 Composants de l'architecture 9
3.5 Gestion des données 10
3.6 Communications entre les microservices 10
3.6.1 Communication synchrone 10
3.6.2 Communication asynchrone 10
3.7 Sécurité 10
3.8 Technologies envisagées 11
3.9 Conclusion 11

4 Conception détaillée des microservices 12
4.1 Introduction 12
4.2 Service commun User/Auth 12
4.2.1 Responsabilités des classes 12
4.2.2 Diagramme de classes 13
4.2.3 Principaux contrats d'API 13
4.3 Microservice Gestion des cours 13
4.3.1 Responsabilités des classes 14
4.3.2 Diagramme de classes 14
4.3.3 Principaux contrats d'API 14
4.4 Microservice Gestion des classes 14
4.4.1 Responsabilités des classes 15
4.4.2 Diagramme de classes 15
4.4.3 Principaux contrats d'API 15
4.5 Microservice Gestion des quiz et examens 15
4.5.1 Responsabilités des classes 16
4.5.2 Diagramme de classes 16
4.5.3 Principaux contrats d'API 16
4.6 Microservice Gestion des réclamations 17
4.6.1 Responsabilités des classes 17
4.6.2 Diagramme de classes 17
4.6.3 Principaux contrats d'API 18
4.7 Microservice Gestion des paiements 18
4.7.1 Responsabilités des classes 18
4.7.2 Diagramme de classes 19

4.7.3 Principaux contrats d'API 19
4.8 Microservice Gestion du forum 19
4.8.1 Responsabilités des classes 19
4.8.2 Diagramme de classes 20
4.8.3 Principaux contrats d'API 20
4.9 Conclusion 20

5 Interactions entre les microservices 21
5.1 Introduction 21
5.2 Résumé des interactions 21
5.3 Communication synchrone : vérifier l'inscription avant un examen 22
5.3.1 Diagramme de séquence synchrone 22
5.4 Communication asynchrone : confirmation d'un paiement 23
5.4.1 Exemple de l'événement PaymentSucceeded 23
5.4.2 Diagramme de séquence asynchrone 23
5.5 Sécurité des communications 24
5.6 Gestion des erreurs et des indisponibilités 24
5.7 Conclusion 24

Table des figures

2.1 Diagramme global de cas d'utilisation de la plateforme e-learning 6

3.1 Architecture globale de la plateforme e-learning 9

4.1 Diagramme de classes du service commun User/Auth 13
4.2 Diagramme de classes du microservice Gestion des cours 14
4.3 Diagramme de classes du microservice Gestion des classes 15
4.4 Diagramme de classes du microservice Gestion des quiz et examens 16
4.5 Diagramme de classes du microservice Gestion des réclamations 18
4.6 Diagramme de classes du microservice Gestion des paiements 19
4.7 Diagramme de classes du microservice Gestion du forum 20

5.1 Vérification synchrone de l'inscription avant une évaluation 22
5.2 Activation asynchrone d'une inscription après un paiement 24

Liste des tableaux

1.1 Microservices prévus pour la plateforme 3

2.1 Acteurs de la plateforme e-learning 4
2.2 Répartition des responsabilités au sein du groupe 7

3.1 Composants de l'architecture proposée 9
3.2 Technologies à confirmer pour le projet 11

4.1 Responsabilités des classes du service commun User/Auth 12
4.2 Contrats d'API principaux du service User/Auth 13
4.3 Responsabilités des classes du service Gestion des cours 14
4.4 Contrats d'API principaux du service Gestion des cours 14
4.5 Responsabilités des classes du service Gestion des classes 15
4.6 Contrats d'API principaux du service Gestion des classes 15
4.7 Responsabilités des classes du service Gestion des examens 16
4.8 Contrats d'API proposés pour le service Gestion des examens 17
4.9 Responsabilités des classes du service Gestion des réclamations 17
4.10 Contrats d'API principaux du service Gestion des réclamations 18
4.11 Responsabilités des classes du service Gestion des paiements 18
4.12 Contrats d'API principaux du service Gestion des paiements 19
4.13 Responsabilités des classes du service Gestion du forum 19
4.14 Contrats d'API principaux du service Gestion du forum 20

5.1 Interactions principales entre les microservices 21

Introduction

L'enseignement en ligne permet de consulter des cours et des ressources pédagogiques à distance. Pour organiser efficacement une formation, il est également nécessaire de gérer les classes, les évaluations, les paiements et la communication entre les différents utilisateurs.

Dans le cadre du projet AWD, notre groupe propose la conception d'une plateforme e-learning destinée aux étudiants, aux enseignants et aux administrateurs. Elle regroupe plusieurs fonctionnalités au sein d'une même application : gestion des cours, des classes, des quiz et des examens, des réclamations, des paiements et du forum.

Le projet est organisé autour de plusieurs microservices. Chaque service prend en charge un domaine fonctionnel. Un service commun `User/Auth`, réalisé collectivement par les membres du groupe, gère l'authentification et les rôles des utilisateurs.

Ce rapport présente principalement la conception de la solution. Il décrit les besoins, l'architecture envisagée, les microservices et leurs interactions. À cette étape intermédiaire, l'objectif est de proposer une conception cohérente ; l'application n'est pas nécessairement finalisée.

Organisation du rapport

Le rapport est organisé comme suit :

- le chapitre 1 présente le projet, sa problématique, ses objectifs et son périmètre ;
- le chapitre 2 décrit les acteurs, les besoins fonctionnels et le diagramme global de cas d'utilisation ;
- le chapitre 3 présente l'architecture globale et les choix techniques envisagés ;
- le chapitre 4 détaille la conception des microservices, leurs entités, leurs diagrammes de classes et leurs principaux contrats d'API ;
- le chapitre 5 présente les interactions entre les microservices, avec des exemples de communications synchrones et asynchrones.

La conclusion résume le travail réalisé et les prochaines étapes du projet.

Chapitre 1

Présentation générale du projet

1.1 Contexte

Les plateformes e-learning donnent accès à des contenus pédagogiques numériques et facilitent l'organisation des formations à distance. Toutefois, une formation nécessite aussi des outils pour gérer les utilisateurs, les cours, les groupes d'étudiants, les évaluations et les échanges avec l'administration.

Notre projet consiste à concevoir une plateforme qui rassemble ces fonctionnalités dans un espace commun. Elle s'adresse principalement aux étudiants, aux enseignants et aux administrateurs.

1.2 Problématique

Lorsque les cours, les inscriptions, les évaluations, les paiements et les demandes d'assistance sont gérés par des outils séparés, les utilisateurs peuvent avoir des difficultés à retrouver leurs informations et à suivre leurs démarches.

La problématique de notre projet est donc la suivante :

Comment concevoir une plateforme e-learning qui centralise les services nécessaires aux étudiants, aux enseignants et aux administrateurs, tout en permettant à chaque utilisateur d'accéder aux fonctionnalités correspondant à son rôle ?

1.3 Solution proposée

Nous proposons une application e-learning organisée autour de plusieurs microservices. Le service commun `User/Auth` prend en charge l'authentification et la gestion des rôles. Les autres services correspondent aux domaines métier de la plateforme.

Table 1.1 – Microservices prévus pour la plateforme

Microservice	**Responsabilité principale**
User/Auth (commun)	Gérer les comptes, l'authentification et les rôles. Ce service est développé collectivement par les membres du groupe.
Gestion des cours	Gérer les cours, les leçons et les ressources pédagogiques.
Gestion des classes	Gérer les classes et les inscriptions des étudiants.
Gestion des quiz et examens	Gérer les évaluations, les questions, les tentatives et les résultats.
Gestion des réclamations	Permettre aux étudiants et aux enseignants de déposer des réclamations, puis de suivre leur traitement et les réponses de l'administration.
Gestion des paiements	Enregistrer les paiements et permettre le suivi de leur état.
Gestion du forum	Permettre aux utilisateurs de participer aux discussions pédagogiques.

1.4 Objectifs du projet

L'objectif général est de proposer un espace numérique qui facilite le suivi des activités pédagogiques et les échanges entre les différents utilisateurs.

Les objectifs spécifiques sont les suivants :

- permettre aux utilisateurs de s'authentifier et d'accéder aux fonctionnalités selon leur rôle ;
- permettre aux étudiants de consulter les cours et les ressources ;
- permettre aux enseignants de gérer leurs cours et leurs activités pédagogiques ;
- organiser les étudiants dans des classes ;
- permettre la création et le passage de quiz et d'examens ;
- permettre le suivi des paiements ;
- permettre aux étudiants et aux enseignants de déposer une réclamation, puis de consulter son statut et les réponses de l'administration ;
- faciliter les échanges pédagogiques à travers un forum.

1.5 Périmètre du projet

La première version du projet porte sur la conception et le développement d'une application web comprenant les services présentés dans le tableau 1.1.

Le projet comprend notamment la gestion des utilisateurs, des cours, des classes, des évaluations, des paiements, des réclamations et du forum. Les acteurs et leurs fonctionnalités détaillées sont présentés dans le chapitre 2.

À cette étape intermédiaire, le rapport se concentre sur la conception générale, la répartition des responsabilités et la définition des principales interactions entre les services. Les choix de technologies et les détails de l'architecture sont présentés dans les chapitres suivants.

Chapitre 2

Analyse des besoins

2.1 Introduction

Ce chapitre présente les acteurs de la plateforme e-learning, les fonctionnalités attendues et les besoins non fonctionnels. Il contient également le diagramme global de cas d'utilisation ainsi que la répartition des responsabilités entre les membres du groupe.

2.2 Acteurs du système

La plateforme est destinée à trois acteurs principaux : l'étudiant, l'enseignant et l'administrateur.

TABLE 2.1 – Acteurs de la plateforme e-learning

Acteur	**Rôle dans la plateforme**
Étudiant	Consulte les cours et les ressources, s'inscrit à une classe, passe des quiz ou des examens, effectue ses paiements, dépose des réclamations et consulte leur état et les réponses reçues. Il peut également participer au forum.
Enseignant	Gère ses cours, ses leçons, ses ressources, ses classes et ses évaluations. Il peut participer au forum et déposer une réclamation.
Administrateur	Gère les utilisateurs et leurs rôles, traite les réclamations, consulte le suivi des paiements et modère le forum.

2.3 Besoins fonctionnels

Les besoins fonctionnels décrivent les principales actions que les utilisateurs peuvent effectuer sur la plateforme.

2.3.1 Authentification et gestion des utilisateurs

Le service commun `User/Auth`, développé par l'ensemble de l'équipe, gère l'authentification et les rôles des utilisateurs.

— permettre aux utilisateurs de se connecter à la plateforme ;

— différencier les rôles étudiant, enseignant et administrateur ;
— limiter l'accès aux fonctionnalités selon le rôle de l'utilisateur ;
— permettre à l'administrateur de gérer les utilisateurs et leurs rôles.

2.3.2 Gestion des cours

— permettre aux enseignants de créer et de modifier des cours ;
— permettre l'ajout, la modification et la suppression de leçons et de ressources ;
— permettre aux étudiants de consulter les cours et leurs ressources ;
— permettre la publication ou le retrait d'un cours.

2.3.3 Gestion des classes

— permettre la création et la gestion des classes ;
— associer une classe à un cours et à un enseignant ;
— permettre aux étudiants de s'inscrire à une classe ;
— permettre la consultation des informations et des séances d'une classe.

2.3.4 Gestion des quiz et examens

— permettre aux enseignants de créer et de gérer des quiz et des examens ;
— permettre l'ajout de questions aux évaluations ;
— permettre aux étudiants autorisés de passer les évaluations ;
— permettre la consultation des résultats.

2.3.5 Gestion des réclamations

— permettre à un étudiant ou à un enseignant de déposer une réclamation à l'aide d'un formulaire ;
— permettre à l'administrateur de consulter et de traiter les réclamations ;
— permettre à l'administrateur de répondre et d'accepter ou de rejeter une réclamation ;
— permettre au demandeur de consulter le statut de sa réclamation et les réponses reçues.

2.3.6 Gestion des paiements

— permettre à l'étudiant d'effectuer un paiement lié à un service de la plateforme ;
— enregistrer les informations et le statut du paiement ;
— permettre à l'étudiant de consulter ses paiements ;
— permettre à l'administrateur de consulter le suivi des paiements.

2.3.7 Gestion du forum

— permettre aux utilisateurs de consulter les discussions ;
— permettre aux étudiants et aux enseignants de participer aux échanges ;
— permettre à l'administrateur de modérer le forum.

2.4 Besoins non fonctionnels

La plateforme doit également répondre aux besoins non fonctionnels suivants :

— **Sécurité :** l'accès aux fonctionnalités doit être contrôlé selon l'identité et le rôle de l'utilisateur ;
— **Confidentialité :** chaque utilisateur ne doit consulter que les informations auxquelles il est autorisé à accéder ;
— **Ergonomie :** l'interface doit être facile à utiliser pour les étudiants, les enseignants et les administrateurs ;
— **Modularité :** chaque microservice doit prendre en charge un domaine fonctionnel défini ;
— **Maintenabilité :** les services doivent pouvoir évoluer sans nécessiter la modification de toute la plateforme ;
— **Fiabilité :** les informations relatives aux cours, aux inscriptions, aux paiements et aux réclamations doivent être conservées de manière cohérente.

2.5 Diagramme global de cas d'utilisation

La figure 2.1 présente les principales interactions entre les acteurs et la plateforme.

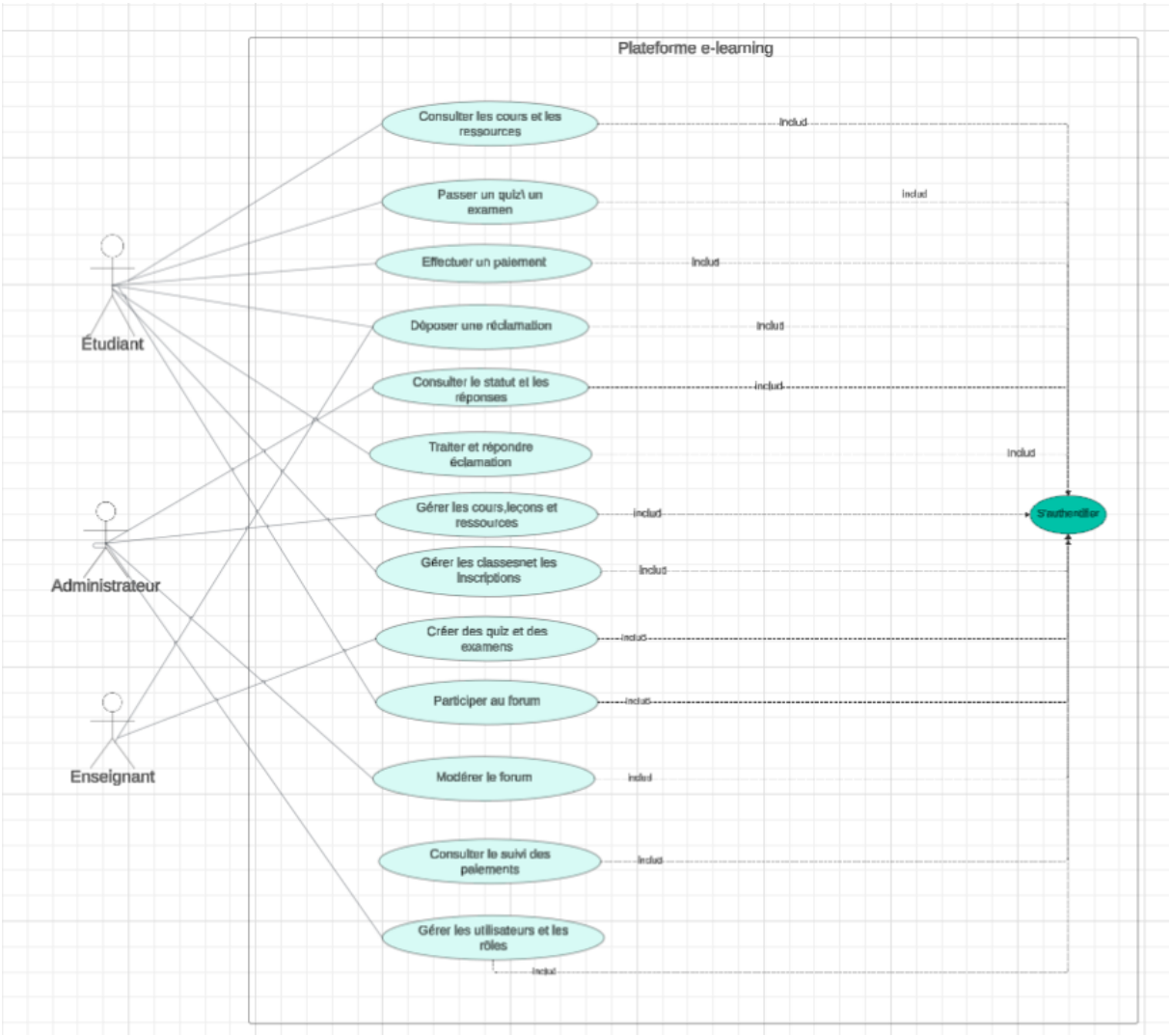


FIGURE 2.1 – Diagramme global de cas d'utilisation de la plateforme e-learning

2.6 Répartition des responsabilités

Le tableau 2.2 présente la répartition des responsabilités entre les membres du groupe. Le service commun `User/Auth` est développé par l'ensemble de l'équipe.

Table 2.2 – Répartition des responsabilités au sein du groupe

Membre	**Responsabilité**	**Contribution**
Maram Dhambri	Gestion des cours	Conception du service des cours, des leçons et des ressources.
Badis Awedi	Gestion des quiz et examens	Conception du service des évaluations, des questions et des résultats.
Aymen Ben Salem	Gestion des réclamations	Conception du service des réclamations et de leur suivi.
Dheker Derouiche	Gestion des classes	Conception du service des classes, des inscriptions et des séances.
Saif Dkhaili	Gestion des paiements	Conception du service des paiements et de leur suivi.
Souleimen Belhadj	Gestion du forum	Conception du service des discussions et des commentaires.
Tous les membres	User/Auth, service commun	Participation collective à la conception du service d'authentification et de gestion des rôles.

2.7 Conclusion

Ce chapitre a présenté les acteurs de la plateforme, les fonctionnalités principales et la répartition des responsabilités dans le groupe. Le chapitre suivant présente l'architecture globale proposée pour organiser les microservices.

Chapitre 3

Architecture globale

3.1 Introduction

La plateforme e-learning est conçue autour d'un frontend web et de plusieurs microservices. Chaque microservice prend en charge un domaine fonctionnel précis. Cette organisation vise à séparer les responsabilités et à faciliter l'évolution des différents modules.

Le frontend communique avec les services backend par l'intermédiaire d'une API Gateway. Le service commun `User/Auth` assure l'authentification et la gestion des rôles.

3.2 Principes de l'architecture

L'architecture proposée repose sur les principes suivants :

- le frontend constitue l'interface utilisée par les étudiants, les enseignants et les administrateurs ;
- l'API Gateway est le point d'entrée des requêtes provenant du frontend et les dirige vers le microservice concerné ;
- le service `User/Auth` est commun à toute la plateforme et gère l'authentification et les rôles ;
- chaque microservice gère les données de son propre domaine ;
- les communications directes entre services utilisent des API REST lorsque le résultat est nécessaire immédiatement ;
- les événements asynchrones peuvent être transmis par un broker de messages, par exemple RabbitMQ, si cette technologie est retenue par l'équipe.

3.3 Diagramme d'architecture globale

La figure 3.1 présente les principaux composants de la plateforme et leurs communications.

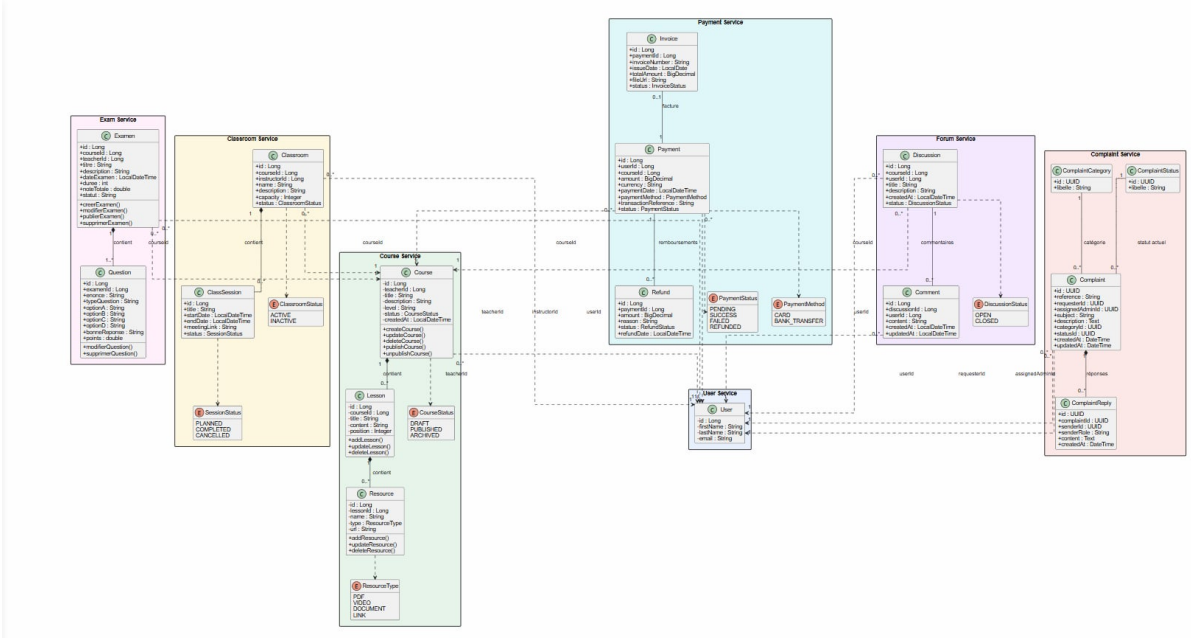


FIGURE 3.1 – Architecture globale de la plateforme e-learning

3.4 Composants de l'architecture

Le tableau 3.1 décrit les composants représentés dans l'architecture globale.

TABLE 3.1: Composants de l'architecture proposée

Composant	**Responsabilité**
Frontend web	Fournit l'interface utilisée par les étudiants, les enseignants et les administrateurs. Il envoie les requêtes à l'API Gateway.
API Gateway	Reçoit les requêtes du frontend et les dirige vers le microservice correspondant. Elle peut également participer à la vérification des jetons et au contrôle d'accès.
User/Auth	Service commun qui gère les utilisateurs, l'authentification et les rôles. Il fournit les informations nécessaires à la gestion des accès.
Gestion des cours	Gère les cours, les leçons et les ressources pédagogiques.
Gestion des classes	Gère les classes, les inscriptions des étudiants et les séances.
Gestion des quiz et examens	Gère les évaluations, les questions, les tentatives et les résultats.
Gestion des réclamations	Gère les formulaires de réclamation, les réponses et le statut des demandes.
Gestion des paiements	Enregistre les paiements, leurs références et leur état. L'intégration à un prestataire réel peut être remplacée par une simulation pendant le développement.
Gestion du forum	Gère les discussions et les commentaires des utilisateurs.

Suite du tableau à la page suivante

Composant	Responsabilité
Bases de données	Chaque microservice possède ses propres données. Les services ne consultent pas directement les tables des autres services.
Broker de messages	Peut transmettre des événements de manière asynchrone entre services. RabbitMQ est une proposition à confirmer par l'équipe.

3.5 Gestion des données

Chaque microservice est responsable des données de son domaine. Par exemple, le service des cours gère ses cours et ses leçons, tandis que le service des réclamations gère les réclamations et leurs réponses.

Les identifiants tels que `userId`, `courseId` et `enrollmentId` peuvent être conservés dans les données d'un service pour faire référence à une information gérée par un autre service. Les services ne doivent cependant pas accéder directement aux tables des autres bases de données. Ils communiquent par API ou par événements.

3.6 Communications entre les microservices

Deux types de communication sont envisagés.

3.6.1 Communication synchrone

La communication synchrone peut utiliser des API REST. Le service qui effectue l'appel attend la réponse du service sollicité.

Exemples :

— le service des quiz et examens peut interroger le service des classes pour vérifier l'inscription d'un étudiant avant une évaluation ;
— le service des classes peut vérifier auprès du service des cours qu'un cours existe ;
— pour une réclamation concernant un paiement, le service des réclamations peut demander au service des paiements de vérifier la référence concernée.

3.6.2 Communication asynchrone

Une communication asynchrone peut utiliser un broker de messages. Le service producteur publie un événement sans attendre que tous les services consommateurs terminent leur traitement.

Par exemple, après la confirmation d'un paiement, le service des paiements peut publier l'événement `PaymentSucceeded`. Le service des classes peut recevoir cet événement et mettre à jour l'inscription correspondante.

Ce mécanisme pourra être implémenté avec RabbitMQ si cette technologie est retenue par l'équipe.

3.7 Sécurité

L'accès aux fonctionnalités doit dépendre du rôle de l'utilisateur. Par exemple, un étudiant consulte ses propres réclamations, tandis que l'administrateur peut traiter les réclamations

qui lui sont accessibles.

Le service User/Auth peut délivrer un jeton JWT après authentification. Ce jeton est transmis avec les requêtes suivantes. Les mots de passe doivent être conservés sous une forme hachée et les informations bancaires sensibles ne doivent pas être stockées par la plateforme.

3.8 Technologies envisagées

Les technologies ci-dessous doivent être confirmées selon les choix réels de l'équipe avant le dépôt final du rapport.

Table 3.2 – Technologies à confirmer pour le projet

Élément	**Technologie retenue par l'équipe**
Frontend	À compléter selon le choix du groupe.
Backend et framework	À compléter selon le choix du groupe.
Base de données	À compléter selon le choix du groupe.
API	REST/JSON proposé ; à confirmer par l'équipe.
Authentification	JWT proposé ; à confirmer par l'équipe.
Messagerie asynchrone	RabbitMQ proposé si un broker est retenu.
Conteneurisation	Docker, si utilisé par l'équipe.

3.9 Conclusion

Ce chapitre a présenté l'architecture globale proposée, les responsabilités des principaux composants, la séparation des données et les modes de communication entre microservices. Le chapitre suivant détaillera la conception des services et leurs entités.

Chapitre 4

Conception détaillée des microservices

4.1 Introduction

La plateforme est composée d'un service commun d'authentification et de plusieurs microservices métier. Chaque microservice gère les entités et les données de son domaine.

Les identifiants tels que `userId`, `courseId` et `enrollmentId` peuvent référencer des informations gérées par un autre microservice. Les services ne consultent pas directement les tables des bases de données des autres services : ils communiquent par API ou, si cette solution est retenue, par événements.

Les endpoints présentés dans ce chapitre sont des contrats proposés pour la conception. Les principaux codes HTTP sont notamment : `200` pour une consultation réussie, `201` pour une création, `202` pour une requête acceptée, `400` pour une requête invalide, `401` pour un utilisateur non authentifié, `403` pour un accès interdit, `404` pour une ressource introuvable et `409` pour un conflit avec l'état actuel de la ressource.

4.2 Service commun User/Auth

Le service `User/Auth` est commun à la plateforme et est conçu collectivement par les membres du groupe. Il gère les comptes, les rôles et l'authentification des utilisateurs. Le diagramme fourni représente les classes `User` et `Role`.

4.2.1 Responsabilités des classes

Table 4.1 – Responsabilités des classes du service commun User/Auth

Classe	**Responsabilité**
`User`	Représenter un compte utilisateur et permettre la gestion de son profil.
`Role`	Définir le rôle fonctionnel d'un utilisateur et participer au contrôle de ses droits d'accès.

4.2.2 Diagramme de classes

La figure 4.1 présente les classes `User` et `Role` du service commun.

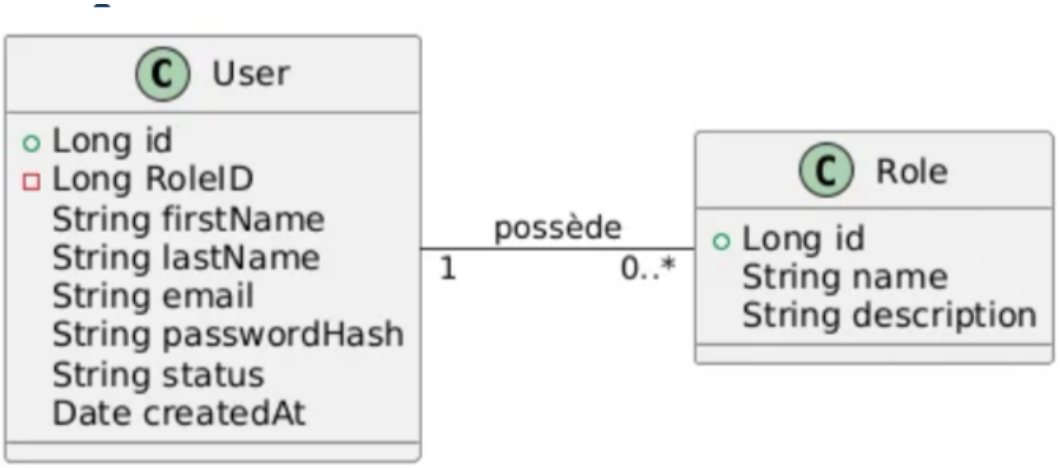


FIGURE 4.1 – Diagramme de classes du service commun User/Auth

4.2.3 Principaux contrats d'API

TABLE 4.2 – Contrats d'API principaux du service User/Auth

Endpoint	**Utilisation et réponses principales**
POST/api/v1/auth/register	Créer un compte utilisateur. Réponses : `201`, `400`, `409` si l'adresse électronique existe déjà.
POST/api/v1/auth/login	Authentifier un utilisateur et retourner son jeton. Réponses : `200`, `400`, `401`.
GET/api/v1/users/me	Consulter les informations de l'utilisateur connecté. Réponses : `200`, `401`.
GET/api/v1/admin/users	Consulter la liste des utilisateurs. Accès réservé à l'administration. Réponses : `200`, `401`, `403`.

4.3 Microservice Gestion des cours

Responsable : Maram Dhambri.

Ce microservice gère les cours, les leçons et les ressources pédagogiques. L'identifiant `teacherId` fait référence à un utilisateur du service User/Auth.

4.3.1 Responsabilités des classes

TABLE 4.3 – Responsabilités des classes du service Gestion des cours

Classe	Responsabilité
`Course`	Représenter un cours et gérer son contenu pédagogique général.
`Lesson`	Organiser le contenu d'un cours en leçons.
`Resource`	Gérer les ressources pédagogiques associées à un cours ou à une leçon.

4.3.2 Diagramme de classes

La figure 4.2 présente les relations entre `Course`, `Lesson` et `Resource`.

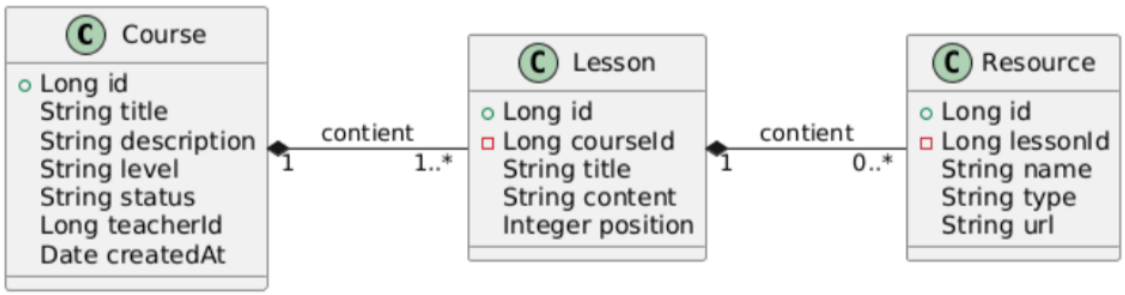


FIGURE 4.2 – Diagramme de classes du microservice Gestion des cours

4.3.3 Principaux contrats d'API

TABLE 4.4 – Contrats d'API principaux du service Gestion des cours

Endpoint	Utilisation et réponses principales
POST/api/v1/courses	Créer un cours. Réponses : `201`, `400`, `401`, `403`.
GET/api/v1/courses	Consulter les cours accessibles. Réponses : `200`, `401`.
GET/api/v1/courses/{id}	Consulter un cours particulier. Réponses : `200`, `401`, `404`.
POST/api/v1/courses/{id}/lessons	Ajouter une leçon à un cours. Réponses : `201`, `400`, `403`, `404`.

4.4 Microservice Gestion des classes

Responsable : Dheker Derouiche.

Ce microservice gère les classes, les inscriptions et les séances. L'identifiant `courseId` fait référence à un cours du service Gestion des cours. Les identifiants des étudiants et des enseignants référencent des utilisateurs du service User/Auth.

4.4.1 Responsabilités des classes

Table 4.5 – Responsabilités des classes du service Gestion des classes

Classe	Responsabilité
`Classroom`	Représenter une classe ou un groupe d'apprenants.
`ClassSession`	Organiser et planifier une séance associée à une classe.
`Enrollment`	Gérer l'inscription d'un apprenant à une classe et suivre son statut.

4.4.2 Diagramme de classes

La figure 4.3 présente les classes du service Gestion des classes.

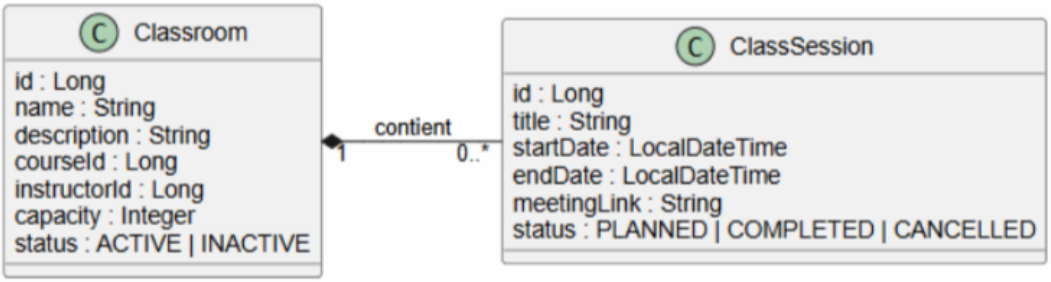


Figure 4.3 – Diagramme de classes du microservice Gestion des classes

4.4.3 Principaux contrats d'API

Table 4.6 – Contrats d'API principaux du service Gestion des classes

Endpoint	Utilisation et réponses principales
POST/api/v1/classes	Créer une classe. Réponses : `201`, `400`, `401`, `403`.
GET/api/v1/classes	Consulter les classes accessibles à l'utilisateur. Réponses : `200`, `401`.
POST/api/v1/classes/{id}/enrollments	Inscrire un étudiant dans une classe. Réponses : `201`, `400`, `403`, `404`, `409`.
GET/api/v1/internal/enrollments/eligibility	Vérifier si un étudiant est inscrit dans une classe. Les identifiants de classe et d'étudiant sont transmis comme paramètres. Réponses : `200`, `404`, `503`.
POST/api/v1/classes/{id}/sessions	Ajouter une séance à une classe. Réponses : `201`, `400`, `403`, `404`.

4.5 Microservice Gestion des quiz et examens

Responsable : Badis Awedi.

Le diagramme fourni pour ce microservice représente la classe `Examen` et la classe `Question`. Un examen contient une ou plusieurs questions. Les identifiants `courseId` et `teacherId` font référence à des données gérées par les services Gestion des cours et User/Auth.

4.5.1 Responsabilités des classes

TABLE 4.7 – Responsabilités des classes du service Gestion des examens

Classe	**Responsabilité**
`Examen`	Créer, modifier, publier et gérer un examen associé à un cours.
`Question`	Définir une question et les éléments nécessaires à son évaluation.

4.5.2 Diagramme de classes

La figure 4.4 présente les classes `Examen` et `Question`, leurs attributs, leurs opérations et leur relation de composition.

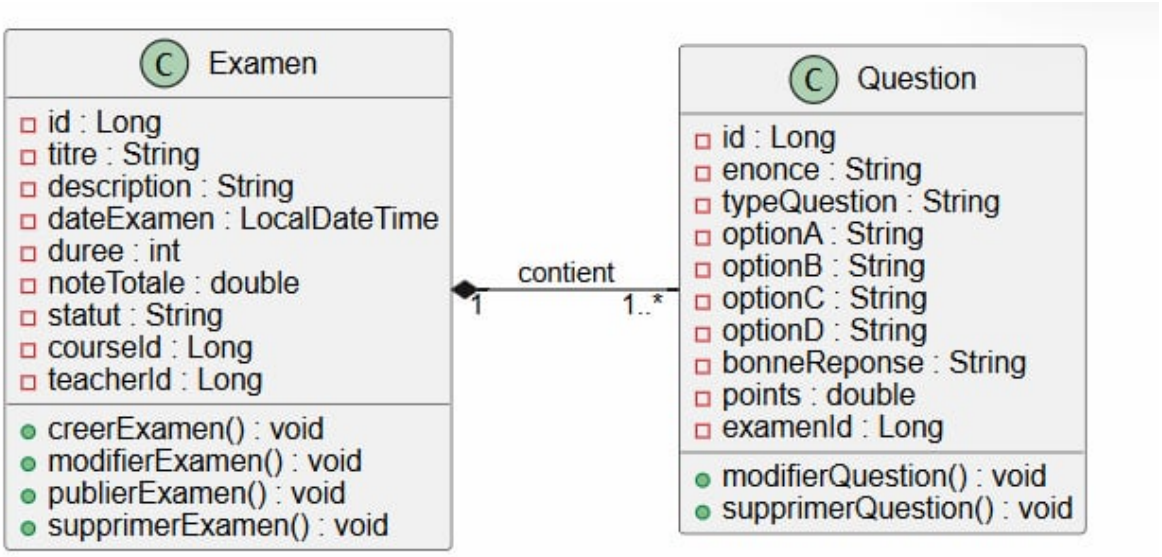


FIGURE 4.4 – Diagramme de classes du microservice Gestion des quiz et examens

4.5.3 Principaux contrats d'API

Les endpoints suivants sont des propositions de conception pour créer, consulter, modifier et publier les examens ainsi que pour gérer leurs questions.

Table 4.8 – Contrats d'API proposés pour le service Gestion des examens

Endpoint	**Utilisation et réponses principales**
POST/api/v1/examens	Créer un examen. Réponses : `201`, `400`, `401`, `403`.
GET/api/v1/examens	Consulter la liste des examens accessibles. Réponses : `200`, `401`.
GET/api/v1/examens/{id}	Consulter un examen et ses questions. Réponses : `200`, `401`, `404`.
PUT/api/v1/examens/{id}	Modifier les informations d'un examen. Réponses : `200`, `400`, `401`, `403`, `404`.
PATCH/api/v1/examens/{id}/publication	Publier un examen. Réponses : `200`, `401`, `403`, `404`, `409`.
DELETE/api/v1/examens/{id}	Supprimer un examen. Réponses : `204`, `401`, `403`, `404`, `409`.
POST/api/v1/examens/{id}/questions	Ajouter une question à un examen. Réponses : `201`, `400`, `401`, `403`, `404`.
PUT/api/v1/questions/{id}	Modifier une question. Réponses : `200`, `400`, `401`, `403`, `404`.
DELETE/api/v1/questions/{id}	Supprimer une question. Réponses : `204`, `401`, `403`, `404`.

Le diagramme fourni représente la définition des examens et de leurs questions. Il ne représente pas les tentatives des étudiants ni leurs réponses enregistrées. Si le scénario de passage d'un examen décrit au chapitre suivant est conservé, il faudra compléter le diagramme avec les classes correspondantes.

4.6 Microservice Gestion des réclamations

Responsable : Aymen Ben Salem.

Ce microservice permet aux étudiants et aux enseignants d'envoyer une réclamation. L'administrateur traite la demande, répond au demandeur et met à jour son statut. Le demandeur peut consulter l'état de sa réclamation et les réponses reçues.

4.6.1 Responsabilités des classes

Table 4.9 – Responsabilités des classes du service Gestion des réclamations

Classe	**Responsabilité**
`ComplaintCategory`	Organiser les réclamations selon leur catégorie.
`ComplaintStatus`	Représenter les différents états possibles d'une réclamation.
`Complaint`	Enregistrer une réclamation et suivre son traitement.
`ComplaintReply`	Gérer les réponses échangées au sujet d'une réclamation.

4.6.2 Diagramme de classes

La figure 4.5 présente les réclamations, les catégories, les statuts et les réponses.

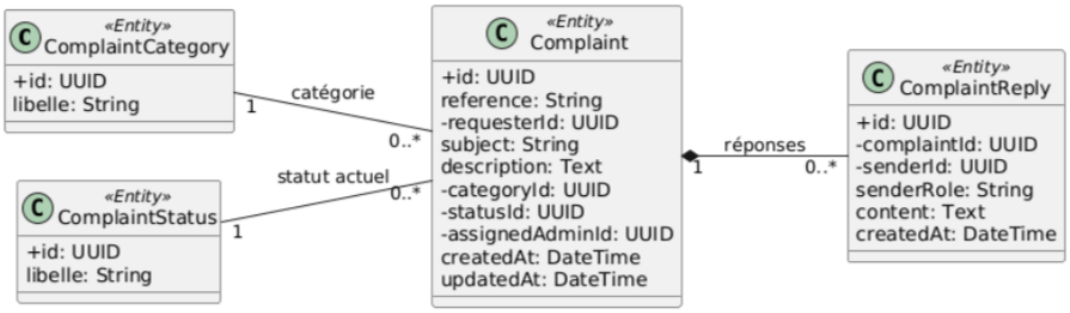


FIGURE 4.5 – Diagramme de classes du microservice Gestion des réclamations

4.6.3 Principaux contrats d'API

TABLE 4.10 – Contrats d'API principaux du service Gestion des réclamations

Endpoint	**Utilisation et réponses principales**
GET/api/v1/complaint-categories	Consulter les catégories proposées dans le formulaire. Réponses : `200`, `401`.
GET/api/v1/complaint-statuses	Consulter les libellés des statuts. Réponses : `200`, `401`.
POST/api/v1/complaints	Créer une réclamation. Le demandeur est déterminé à partir du JWT. Réponses : `201`, `400`, `401`, `404`.
GET/api/v1/complaints	Consulter ses réclamations et leur statut. Réponses : `200`, `401`.
POST/api/v1/complaints/{id}/replies	Ajouter une réponse à une réclamation. Réponses : `201`, `401`, `403`, `404`, `409`.
PATCH/api/v1/admin/complaints/{id}/status	Modifier le statut d'une réclamation. Accès réservé à l'administrateur. Réponses : `200`, `400`, `403`, `404`, `409`.

4.7 Microservice Gestion des paiements

Responsable : Saif Dkhaili.

Ce microservice enregistre les paiements, les factures et les remboursements. Il ne doit pas conserver les numéros de cartes bancaires. L'identifiant `enrollmentId` peut référencer une inscription gérée par le service Gestion des classes.

4.7.1 Responsabilités des classes

TABLE 4.11 – Responsabilités des classes du service Gestion des paiements

Classe	**Responsabilité**
`Invoice`	Représenter une facture liée à un achat ou à un service.
`Payment`	Enregistrer et suivre une transaction de paiement.
`Refund`	Gérer une demande de remboursement liée à un paiement.

4.7.2 Diagramme de classes

La figure 4.6 présente les factures, les paiements et les remboursements.

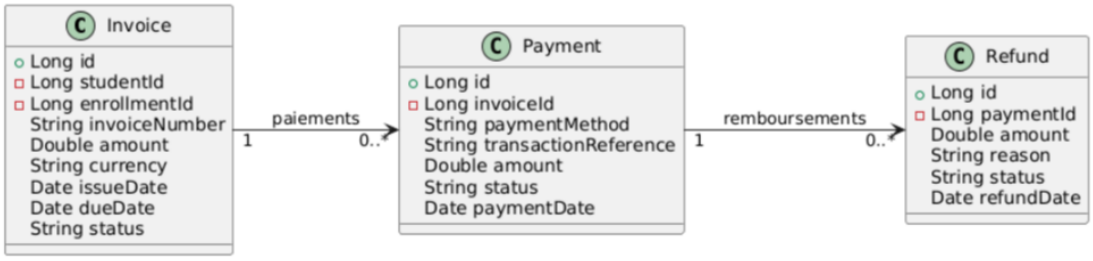


FIGURE 4.6 – Diagramme de classes du microservice Gestion des paiements

4.7.3 Principaux contrats d'API

TABLE 4.12 – Contrats d'API principaux du service Gestion des paiements

Endpoint	**Utilisation et réponses principales**
POST/api/v1/payments	Créer une demande de paiement. Réponses : `202`, `400`, `401`.
GET/api/v1/payments/{id}	Consulter un paiement auquel l'utilisateur est autorisé à accéder. Réponses : `200`, `401`, `403`, `404`.
GET/api/v1/internal/payments/{id}/complaint-context	Fournir au service Réclamations les informations nécessaires pour vérifier un paiement lié à une demande. Réponses : `200`, `404`, `503`.
POST/api/v1/payments/{id}/refunds	Créer une demande de remboursement. Accès soumis aux autorisations de l'administration. Réponses : `201`, `401`, `403`, `404`, `409`.

4.8 Microservice Gestion du forum

Responsable : Souleimen Belhadj.

Le diagramme fourni représente les catégories, les publications et les commentaires. Une publication (`Post`) correspond à une discussion du forum. Les identifiants `userId` font référence aux utilisateurs du service User/Auth.

4.8.1 Responsabilités des classes

TABLE 4.13 – Responsabilités des classes du service Gestion du forum

Classe	**Responsabilité**
`Category`	Organiser les sujets du forum par catégorie.
`Post`	Représenter une publication ou un sujet de discussion.
`Comment`	Gérer les commentaires associés à une publication.

4.8.2 Diagramme de classes

La figure 4.7 présente les relations entre les catégories, les publications et les commentaires.

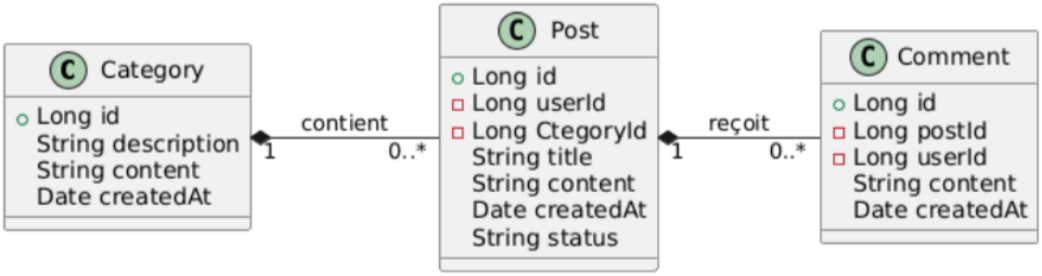


FIGURE 4.7 – Diagramme de classes du microservice Gestion du forum

4.8.3 Principaux contrats d'API

TABLE 4.14 – Contrats d'API principaux du service Gestion du forum

Endpoint	**Utilisation et réponses principales**
GET/api/v1/categories	Consulter les catégories du forum. Réponses : `200`, `401`.
POST/api/v1/posts	Créer une publication dans une catégorie. Réponses : `201`, `400`, `401`, `403`.
GET/api/v1/posts	Consulter la liste des publications accessibles. Réponses : `200`, `401`.
GET/api/v1/posts/{id}	Consulter une publication et ses commentaires. Réponses : `200`, `401`, `404`.
POST/api/v1/posts/{id}/comments	Ajouter un commentaire à une publication. Réponses : `201`, `400`, `401`, `403`, `404`.

4.9 Conclusion

Ce chapitre a présenté les responsabilités des classes, les diagrammes et les contrats d'API proposés pour les microservices de la plateforme. Le chapitre suivant décrit les interactions entre ces services à l'aide d'exemples de communications synchrones et asynchrones.

Chapitre 5

Interactions entre les microservices

5.1 Introduction

Les microservices doivent échanger des informations pour réaliser certains scénarios de la plateforme. Deux formes de communication sont prévues :

- la communication **synchrone**, généralement réalisée par API REST lorsque le service demandeur a besoin d'une réponse immédiate ;
- la communication **asynchrone**, réalisée par la transmission d'événements lorsque le traitement peut continuer sans réponse immédiate.

Chaque service conserve ses propres données. Les échanges entre services passent par des API ou des événements ; aucun service ne lit directement la base de données d'un autre service.

5.2 Résumé des interactions

Le tableau 5.1 résume quelques interactions proposées pour la plateforme.

Table 5.1 – Interactions principales entre les microservices

Service demandeur	**Service destinataire**	**Mode**	**Objectif**
Gestion des classes	Gestion des cours	REST synchrone	Vérifier qu'un cours existe avant de créer une classe qui lui est associée.
Quiz et examens	Gestion des classes	REST synchrone	Vérifier qu'un étudiant est inscrit à une classe avant de l'autoriser à commencer une évaluation.
Gestion des réclamations	Gestion des paiements	REST synchrone	Vérifier qu'un paiement lié à une réclamation existe et correspond au demandeur.
Gestion des paiements	Gestion des classes	Événement asynchrone	Après la confirmation d'un paiement, informer le service Classes pour qu'il active l'inscription correspondante.

5.3 Communication synchrone : vérifier l'inscription avant un examen

Avant de commencer un examen, le service Quiz et examens doit vérifier auprès du service Gestion des classes que l'étudiant est bien inscrit dans la classe concernée.

Le scénario proposé est le suivant :

1. l'étudiant demande à commencer une évaluation depuis le frontend ;
2. l'API Gateway transmet la demande au service Quiz et examens ;
3. ce service interroge le service Gestion des classes avec les identifiants de la classe et de l'étudiant ;
4. le service Classes répond en indiquant si l'inscription est active ;
5. si l'étudiant est inscrit, le service d'évaluation crée une tentative ;
6. dans le cas contraire, l'accès à l'évaluation est refusé.

L'appel interne peut utiliser l'endpoint suivant :

```
GET /api/v1/internal/enrollments/eligibility
```

Les identifiants `classId` et `studentId` sont transmis au service Classes. Le service d'évaluation ne consulte pas directement la base de données des classes.

5.3.1 Diagramme de séquence synchrone

La figure 5.1 illustre le scénario de vérification d'inscription avant le début d'une évaluation.

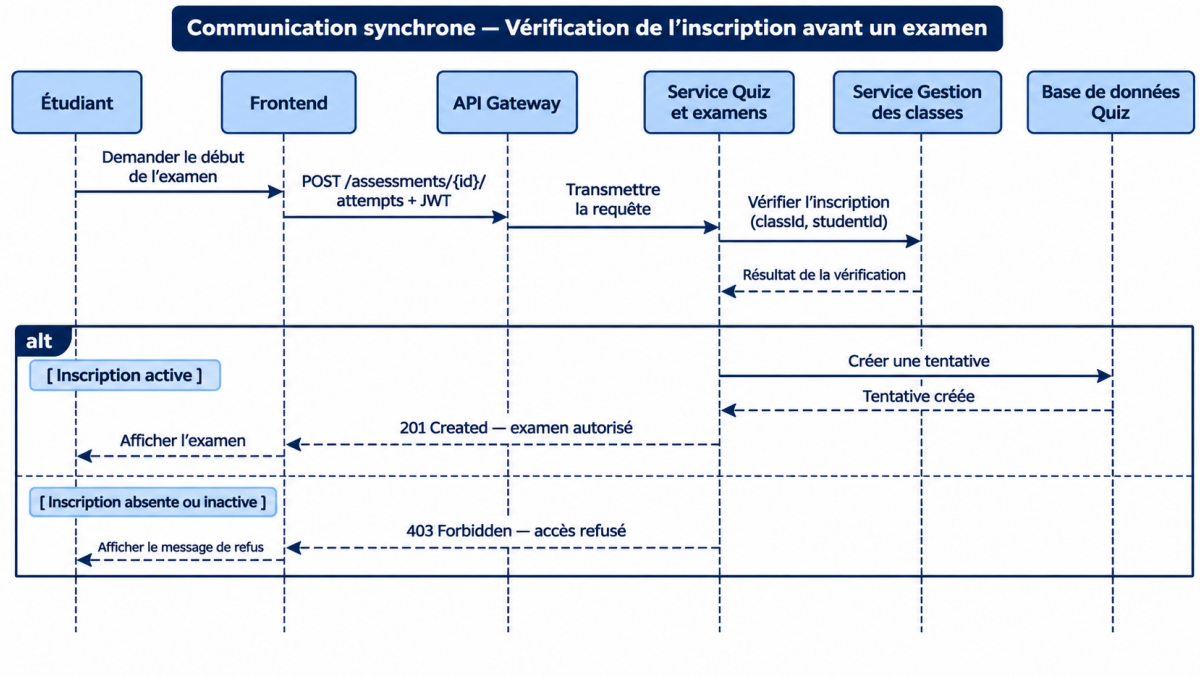


FIGURE 5.1 – Vérification synchrone de l'inscription avant une évaluation

5.4 Communication asynchrone : confirmation d'un paiement

Lorsqu'un étudiant effectue un paiement pour une inscription, celle-ci peut rester temporairement à l'état `PENDING_PAYMENT`. Quand le paiement est confirmé, le service Paiements publie l'événement `PaymentSucceeded`. Le service Classes reçoit cet événement et active l'inscription correspondante.

Le scénario est le suivant :

1. le frontend demande la création d'un paiement ;
2. le service Paiements enregistre la demande avec un état en attente ;
3. le service Paiements retourne une réponse indiquant que la demande est prise en charge ;
4. après la confirmation du paiement, le service Paiements publie `PaymentSucceeded` ;
5. le service Classes consomme l'événement ;
6. si l'inscription correspondante existe, son état devient `ACTIVE`.

Le service Paiements et le service Classes n'ont pas besoin d'être en communication directe pendant tout le traitement. Si RabbitMQ est retenu par l'équipe, il peut servir de broker pour transmettre l'événement.

5.4.1 Exemple de l'événement PaymentSucceeded

```
{
  "eventId": "event-123",
  "eventType": "PaymentSucceeded",
  "occurredAt": "2026-04-12T10:30:00Z",
  "data": {
    "paymentId": "payment-001",
    "enrollmentId": "enrollment-015",
    "studentId": "user-027",
    "amount": 150.00,
    "currency": "EUR"
  }
}
```

L'identifiant `eventId` peut aider le service consommateur à éviter de traiter deux fois le même événement. Le service Classes vérifie `enrollmentId`, puis met à jour son propre enregistrement d'inscription.

5.4.2 Diagramme de séquence asynchrone

La figure 5.2 illustre la publication et la consommation de l'événement de confirmation de paiement.

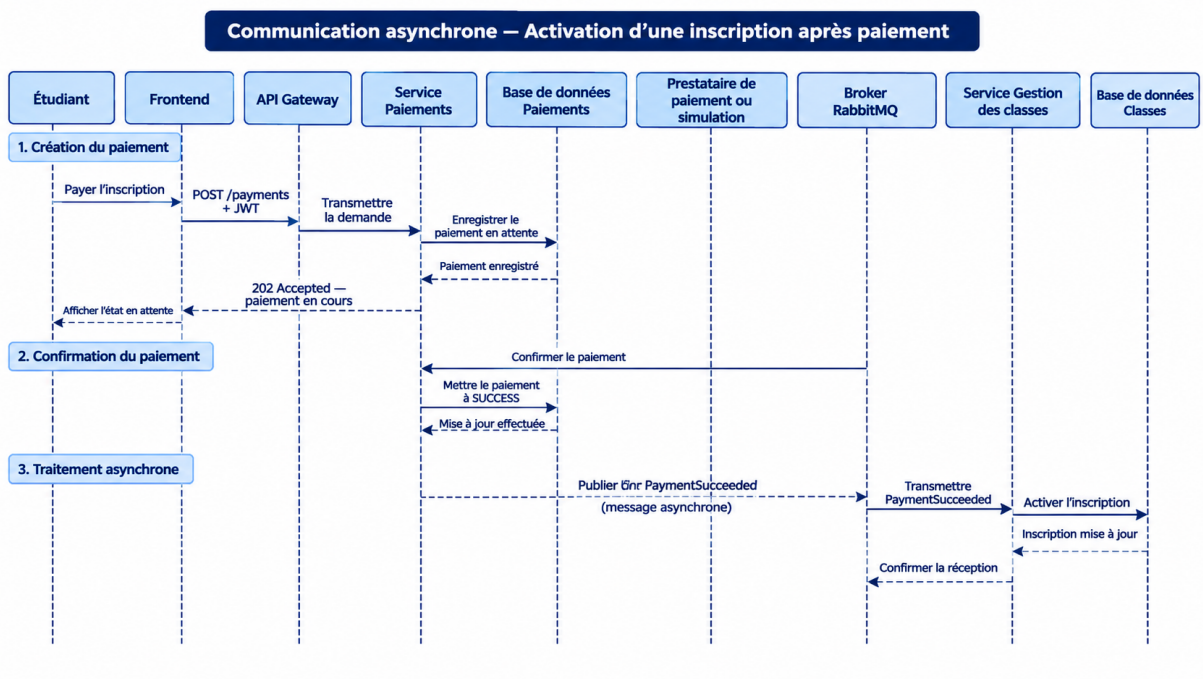


FIGURE 5.2 – Activation asynchrone d'une inscription après un paiement

5.5 Sécurité des communications

Les communications doivent respecter les règles suivantes :

- les requêtes du frontend passent par l'API Gateway et utilisent un jeton JWT ;
- les services vérifient les droits nécessaires avant d'exécuter une opération ;
- un étudiant ne consulte que ses propres réclamations et paiements ;
- les endpoints internes utilisés entre microservices ne sont pas destinés à être appelés directement par les utilisateurs ;
- les données sensibles, telles que les mots de passe ou les numéros de carte bancaire, ne doivent pas être transmises ou stockées sans protection.

5.6 Gestion des erreurs et des indisponibilités

Lorsqu'un service externe nécessaire à une opération ne répond pas, le service demandeur doit éviter de produire un résultat incohérent. Par exemple, si le service Paiements ne répond pas lors de la vérification d'un paiement lié à une réclamation, la création de la réclamation peut être refusée temporairement avec une réponse `503 Service Unavailable`.

Pour les événements asynchrones, le service consommateur doit traiter les messages de manière fiable et éviter d'appliquer plusieurs fois une même opération si un événement est reçu en double.

5.7 Conclusion

Ce chapitre a présenté les principales interactions synchrones et asynchrones entre les microservices. La vérification d'une inscription avant un examen illustre un appel REST synchrone, tandis que l'événement `PaymentSucceeded` illustre une communication asyn-

chrone entre les services Paiements et Classes. Ces échanges respectent la séparation des données entre microservices.